Deep Learning: GANs, Autoencoders, Attention

M. Vazirgiannis

Data Science and Mining Team (DASCIM), LIX
École Polytechnique
http://www.lix.polytechnique.fr/dascim
Google Scholar: https://bit.ly/2rwmvQU

October, 2025

”

Outline

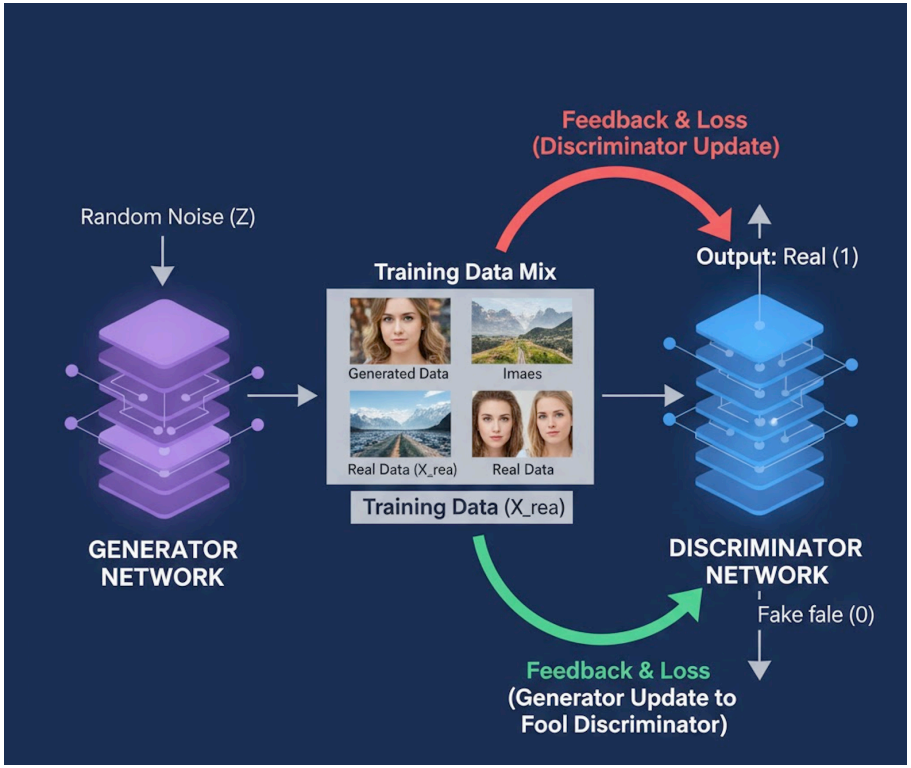
1. Generative Adversarial Networks (GANs)
2. Auto-encoders for unsupervised learning
3. Attention based architectures

GANs (Generative Adversarial Networks)— Motivation

- Deep learning models are great at learning from data.
- But what if we want to generate new data that looks real?
- GANs (Generative Adversarial Networks) provide a framework for learning to generate realistic data samples.
- *Unsupervised* ability to generate highly realistic, novel data that closely matches a training distribution
- Applications: image synthesis, super-resolution, style transfer, and data augmentation.

GAN Architecture Overview

- Two neural networks compete:
  - **Generator (G)**: learns to generate fake samples that resemble the training data.
  - **Discriminator (D)**: learns to distinguish between real and fake samples.
- Training is a **minimax game** between G and D.



@Gemini

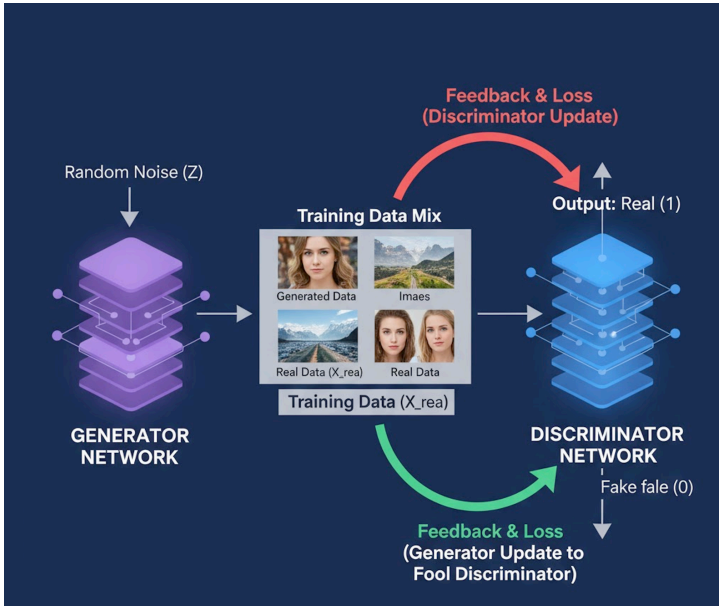
GAN Architecture Overview

Generator Network (GN)

- **Input:** GN (purple stack) generates a random noise vector, Z, as input/seed.
- **Output:** Generator transforms Z to data intended to resemble real training data: **Generated Data**: X_{fake} (the fake images).

Discriminator Network (DN)

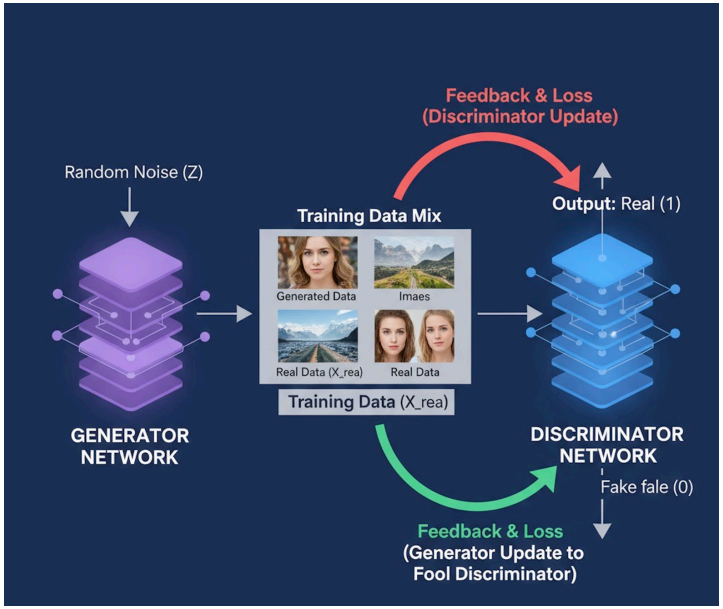
- **Input Mix:** DN Network (blue stack) receives mixture of two data types: Generated Data: X_{fake} from Generator and Real Data: X_{real} directly from training dataset.
- **Output:** DN aims to classify input: Output: **Real (1)**: input comes from real training data (X_{real}) vs. Output: **Fake (0)**: input was generated (X_{fake}).



@Gemini

GAN Architecture Overview

Feedback Loops (Adversarial Training): **training process** driven by loss and feedback from Discriminator's output, updates both networks:



A. Feedback to Discriminator (Red Arrow)

- **Goal:** Train Discriminator to become better at distinguishing real from fake data.
- **Mechanism:** *Feedback & Loss (Discriminator Update)* (red arrow) computed based on how accurately the Discriminator classified mixed data.
- If misclassifies real data as fake, or fake data as real, loss is high, and weights are updated to improve classification accuracy.

B. Feedback to Generator (Green Arrow)

- **Goal:** Train Generator to become better at fooling the Discriminator.
- **Mechanism:** *Feedback & Loss (Generator Update to Fool Discriminator)* (green arrow) uses only Discriminator's result on the Generated Data (X_{fake}).
- If Discriminator successfully identifies the generated image as fake (outputs '0'), Generator receives a large loss signal.

GANs - Mathematical Formulation

The objective function for the standard GAN is the minimax game defined by the V(D, G) :

$$\min_G \max_D V(D,G) = \mathbb{E}_{x\sim p_{\text{data}}(x)}[\log D(x)] + \mathbb{E}_{z\sim p_z(z)}[\log(1 - D(G(z)))]$$

- $\min_G \max_D$: Represents the adversarial goal: Generator (G) aims to minimize the function (performance of D), while the Discriminator (D) aims to maximize it.
- $\mathbb{E}_{x\sim p_{\text{data}}(x)}$: expected value over all real data instances x sampled from the data distribution $p_{data}(x)$.
- $\log D(x)$: The term Discriminator maximizes ($D(x)1$).
- $\mathbb{E}_{z\sim p_z(z)}$: expected value over all noise vectors z sampled from the noise distribution $p_z(z)$.
- $\log(1 - D(G(z)))$: term the Discriminator maximizes (wants $D(G(z)) \approx 0$) and the Generator minimizes (wants $D(G(z)) \approx 1$).

GAN Challenges: Training Instability and Mode Collapse

- **Adversarial Objective:**

$$\min_G \max_D V(D,G) = \mathbb{E}_{x\sim p_{\text{data}}}[\log D(x)] + \mathbb{E}_{z\sim p_z}[\log(1 - D(G(z)))]$$

  The generator G tries to minimize this value while the discriminator D maximizes it.

- **Training Instability:**
  - Non-convex, saddle-point optimization $\Rightarrow$ oscillations, divergence.
  - Sensitive to the relative learning rates of G and D.
  - If D converges too fast, G's gradients vanish.
- **Mode Collapse:**
  - G finds a few "modes" that fool D well, ignoring others.
  - Mathematically, $p_G(x)$ collapses to a subset of modes in $p_{\text{data}}(x)$.
  - Leads to low diversity — e.g., identical faces, digits, or objects.

Improvements: Wasserstein GAN (WGAN)

- **Key Idea:** Replace GAN loss with the *Earth Mover's (Wasserstein-1) distance*:

$$W(p_{\text{data}}, p_G) = \inf_{\gamma \in \Pi(p_{\text{data}}, p_G)} \mathbb{E}_{(x,y)\sim\gamma}[\|x - y\|]$$

- **WGAN Objective:**

$$\min_G \max_{D \in \mathcal{D}} \mathbb{E}_{x\sim p_{\text{data}}}[D(x)] - \mathbb{E}_{z\sim p_z}[D(G(z))]$$

  where $\mathcal{D}$ is the set of 1-Lipschitz functions.

- **Implementation:**
  - Enforce 1-Lipschitz constraint via weight clipping or gradient penalty.
  - Provides smoother gradients $\Rightarrow$ stable convergence.
- **Advantages:**
  - Loss correlates with sample quality.
  - Less sensitive to architecture and hyper-parameters.

Further Improvements: cGANs, StyleGAN, Diffusion Models

- **Conditional GANs (cGANs):**

$$\min_G \max_D \mathbb{E}_{x,y}[\log D(x|y)] + \mathbb{E}_{z,y}[\log(1 - D(G(z|y)))]$$

  Conditioning on label y guides generation, improves diversity/control.
- **StyleGAN:**
  - Introduces mapping network $f(z) \to w$ for disentangled latent space.
  - Adaptive Instance Normalization (AdaIN):

$$\text{AdaIN}(x_i, y) = y_s \frac{x_i - \mu(x_i)}{\sigma(x_i)} + y_b$$

    Enables fine-grained control over style (e.g., pose, texture, expression).
- **Diffusion-based Models:**
  - Replace adversarial loss with probabilistic denoising:

$$q(x_t|x_{t-1}) = \mathcal{N}(x_t; \sqrt{1-\beta_t}x_{t-1}, \beta_t I)$$

  - Model learns to reverse this process:

$$p_\theta(x_{t-1}|x_t) = \mathcal{N}(x_{t-1}; \mu_\theta(x_t, t), \Sigma_\theta(x_t, t))$$

  - Results: More stable training, higher fidelity, and diversity.

Summary: GAN Challenges and Improvements

Main Challenges

- Training instability due to non-convex adversarial loss.
- Mode collapse — generator ignores parts of the data distribution.

Key Advances

- **WGAN:** Stable and interpretable optimization via Wasserstein distance.
- **cGANs:** Conditional control reduces mode collapse.
- **StyleGAN:** Hierarchical latent control, state-of-the-art image quality.
- **Diffusion Models:** Non-adversarial training, superior sample diversity.

Applications of GANs

- **Image-to-Image Translation**
  - Learn mapping between two domains $\mathcal{X} \leftrightarrow \mathcal{Y}$.
  - Objective (Pix2Pix):

$$\mathcal{L}_{\text{Pix2Pix}} = \mathbb{E}_{x,y}[\log D(x,y)] + \mathrm{E}_{x,z}[\log(1 - D(x, G(x,z)))] + \lambda \|y - G(x,z)\|_1$$

  - Example: Edges $\rightarrow$ Photos, Day $\leftrightarrow$ Night, Maps $\leftrightarrow$ Aerial images.
- **Super-Resolution (SRGAN)**
  - Generates high-resolution (HR) images from low-resolution (LR) inputs.

$$\mathcal{L}_{\text{SRGAN}} = \mathcal{L}_{\text{adv}} + \eta \mathcal{L}_{\text{content}}$$

  - $\mathcal{L}_{\text{content}}$ uses perceptual VGG features for sharpness
- **Text-to-Image Generation (e.g., AttnGAN, DALL·E)**
  - Conditioning on text embedding e_t:

$$G : (z, e_t) \rightarrow \text{Image}$$

  - Attention mechanisms align visual features with words.
  - Output: images aligned with text semantics.
- **Deepfake Synthesis**
  - Face-swapping using identity-preserving loss:

$$\mathcal{L}_{\text{id}} = \| f_{\text{enc}}(x_{\text{real}}) - f_{\text{enc}}(G(x_{\text{src}}, x_{\text{tgt}}))$$

  - Applications: film dubbing, AR filters, synthetic media (w/ ethical constraints).

Key Takeaways

- GANs learn by competition between Generator and Discriminator.
- Capable of generating realistic and high-dimensional data.
- Still challenging to train — careful design and tuning required.
- Foundation for modern generative models like StyleGAN and diffusion models.

References

- Goodfellow et al., “Generative Adversarial Nets,” NIPS 2014.
- Ian Goodfellow’s GAN tutorial: https://www.iangoodfellow.com
- MIT Deep Learning Lecture on GANs.

Outline

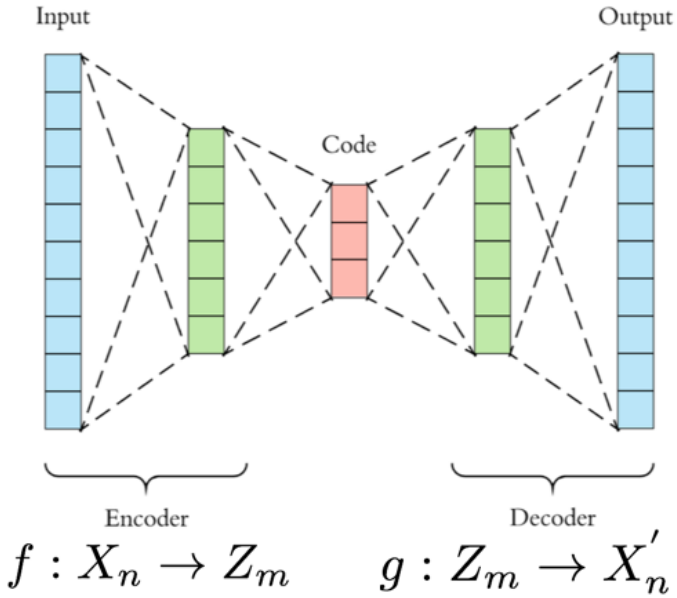
1. Generative Adversarial Networks (GANs)
2. Auto-encoders for unsupervised learning
3. Attention based architectures

Autoencoders - the concept

- introduced by Hinton
  - address the problem of “backpropagation without a teacher”
  - Need to formulate an error function – using input data as the teacher [RUM1986]
- more recently, “deep architecture” approach (Hinton et al., 2006; Hinton and Salakhutdinov, 2006; Bengio and LeCun, 2007;Erhan et al., 2010)
  - Restricted Boltzmann Machines (RBMS), stacked, trained bottom up in unsupervised fashion.

Autoencoders

- Assume a set of n-dim data X_n - we assume transformations f, g
  - f : mapping data to a lower dim space Z_m (code)
  - g: mapping Z_m to the an original dim data $X'_n \sim X_n$
- Learning: minimize the loss $L(x(f(g(x))$
  - $|code| < |input|$: undercomplete AE
  - Sparse autoencoders: $L(x(f(g(x)) + \Omega(Z_m)$



Applications of AEs

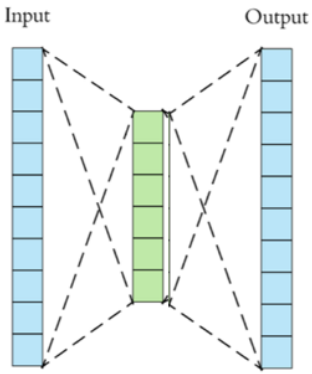
- Learn a low dimensional Z_m representation of the data X
  - learning low dimensional space with less reconstruction error than PCA (Hinton 2006)
  - Features used in classification and improve generalization (Hinton 2007)
- Perform clustering – unsupervised learning
- Information Retrieval
  - Semantic hashing for images and NLP (Hinton 2007, Torralba 2007)
- Learn data features in the absence of a supervised task

Sparse Autoencoders (AE)

- Assume AE: n input neurons, m neurons in hidden layer and k training data
- Assume $a_j^{(2)}(x^{(i)})$ activation of j-th neuron in hidden layer for x(i)-th data point
- average output of j-th neuron of the hidden layer after data training: $\hat{\rho} = \frac{1}{k}\sum_i^k (a_j^{(2)}(x^{(i)}))$
- Assume only few hidden layer neurons should fire for each training sample
  - output ρ of a neuron should be small (i.e. 0.02)
  - Penalize $\hat{\rho}$ deviating significantly from ρ:

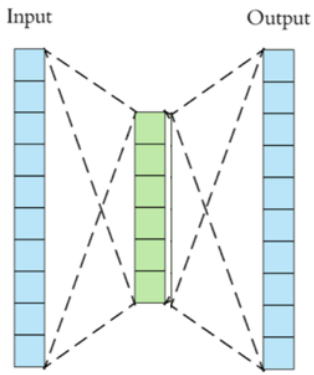
$$\sum_{j=1}^{m} \rho log \frac{\rho}{\hat{\rho}_j} + (1-\rho) log \frac{1-\rho}{1-\hat{\rho}_j} = \sum_{j=1}^{m} KL(\rho||\hat{\rho}_j)$$

  - KL divergence metric of distribution difference – sparsity term



Sparse Autoencoders (AE)

- The overall cost function:
  $J_{sparse}(W, b) = J(W, b) + \beta \sum_{j=1}^{m} KL(\rho||\hat{\rho}_j)$
- Need to incorporate the KL divergence in the back-propagation
- The back-prop at the second (hidden) layer:
  $\delta_i^{(2)} = (\sum_{j=1}^{m} W_{ji}^{(2)} \delta_j^{(3)}) f'(z_i^{(2)})$
- Take into account sparsity:
  $\delta_i^{(2)} = (\sum_{j=1}^{m} W_{ji}^{(2)} \delta_j^{(3)}) + \beta(-\frac{\rho}{\hat{\rho}_i} + \frac{1-\rho}{1-\hat{\rho}_i}) f'(z_i^{(2)}))$
- Images that enable hidden units are image fundamental features (i.e. basic shapes)



Auto-encoders for clustering [Hie te al 2016]

Deep Embedding Clustering (DEC)

- Assume data X pf n points, cluster them into k. clusters in a lower dimensional space via a nonlinear mapping $f_\theta : X \rightarrow Z$, Z, θ: code, trainable parameters
- learning simultaneously i. the center of the k clusters in the Z space and μ,θ
- DEC phases:
  - parameter initialization with a deep stacked autoencoder (Vincent 2010)
  - parameter optimization (i.e., clustering), where we iterate between computing an auxiliary target distribution and minimizing the Kullback–Leibler (KL) divergence to μ

Auto-encoders for clustering [Hie te al 2016]

- soft assignment of embedded point $z_i = f_\theta(x_i)$ and the cluster centroids μ_j

$$q_{ij} = \frac{(1 + ||z_i - \mu_j||^2/\alpha)^{-\frac{\alpha+1}{2}}}{\sum_{j'} (1 + ||z_i - \mu_{j'}||^2/\alpha)^{-\frac{\alpha+1}{2}}}$$

- iteratively refine clusters by learning from their high confidence assignments with the help of an auxiliary target distribution

$$p_{ij} = \frac{q_{ij}^2/f_j}{\sum_{j'} q_{ij'}^2/f_{j'}}$$

where $f_j = \sum_i q_{ij}$are the soft cluster frequencies.

- Then the error is $L = KL(P||Q) = \sum_i \sum_j p_{ij} log \frac{p_{ij}}{q_{ij}}$

Auto-encoders for clustering [Hie te al 2016]

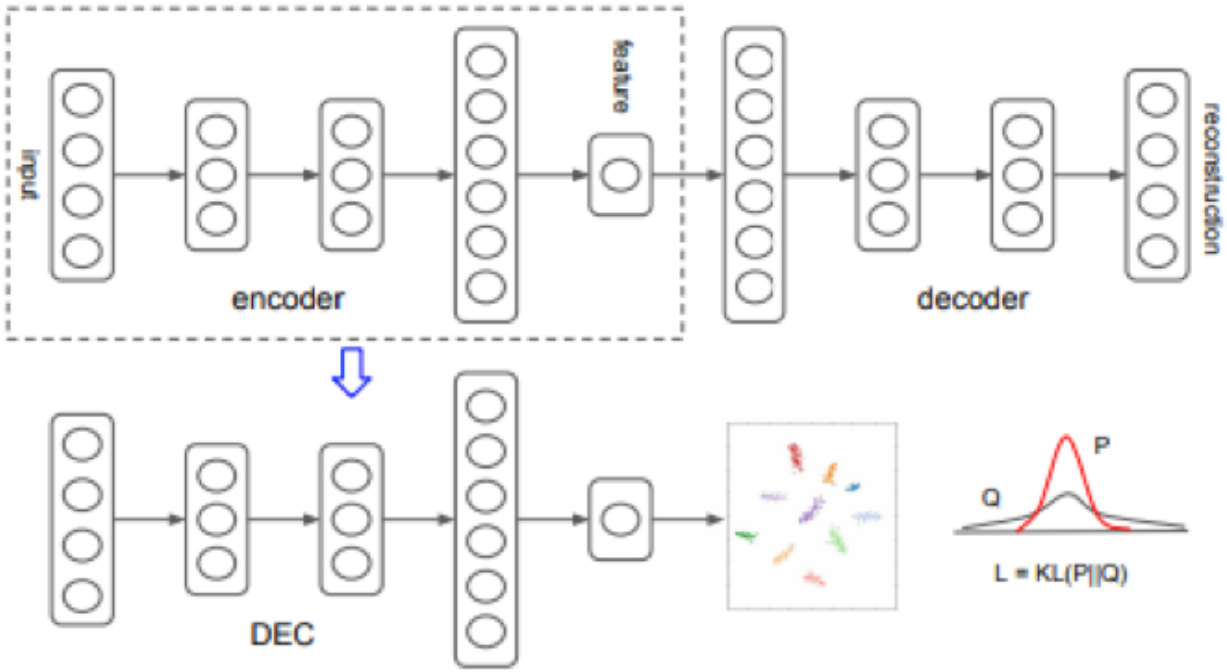
- jointly optimize cluster centers $\{\mu_j\}$ and DNN parameters θ using Stochastic Gradient Descent (SGD):

$$\frac{\partial L}{\partial z_i} = \frac{\alpha+1}{\alpha} \sum_j (1 + \frac{||z_i - \mu_j||^2}{\alpha})^{-1} \times (p_{ij} - q_{ij})(z_i - \mu_j)$$

$$\frac{\partial L}{\partial \mu_j} = -\frac{\alpha+1}{\alpha} \sum_i (1 + \frac{||z_i - \mu_j||^2}{\alpha})^{-1} \times (p_{ij} - q_{ij})(z_i - \mu_j)$$

- gradients $\frac{\partial L}{\partial z_i}$ are passed down to the DNN
- gstandard backpropagation compute DNN's parameter gradient $\frac{\partial L}{\partial \theta}$
- discovering cluster assignments, stop when less than tol % of points change cluster assignment between consecutive iterations.

Auto-encoders for clustering [Hie te al 2016]



To initialize the cluster centers:

- Pass data through the initialized DNN to get embedded data points and then perform standard k-means clustering in the feature space Z to obtain k initial centroids $\{\mu_j\}_{j=1}^{k}$.

Auto-encoders for clustering [Hie te al 2016]

Dataset	# Points	# classes	Dimension	% of largest class
MNIST (LeCun et al., 1998)	70000	10	784	11%
STL-10 (Coates et al., 2011)	13000	10	1428	10%
REUTERS-10K	10000	4	2000	43%
REUTERS (Lewis et al., 2004)	685071	4	2000	43%

Method	MNIST	STL-HOG	REUTERS-10k	REUTERS
k-means	53.49%	28.39%	52.42%	53.29%
LDMGI	84.09%	33.08%	43.84%	N/A
SEC	80.37%	30.75%	60.08%	N/A
DEC w/o backprop	79.82%	34.06%	70.05%	69.62%
DEC (ours)	**84.30%**	**35.90%**	**72.17%**	**75.63%**

Auto-encoders for clustering [Hie te al 2016]



(a) MNIST



(b) STL-10

Top 10 results for each of the 10 clusers for the MNIST and STL data sets

Why is Unsupervised learning important

- Machine learning: learning representations (features, latent variables) to capture the statistical dependencies
- Supervised learning: from input variables to output variables, – i.e. categories
  - supervised learning require ”teaching” computer concepts that matter to humans
  - supervised deep learning discover meaningful intermediate representations in their layers.
- Unsupervised learning
  - capture all possible dependencies between any subset of variables
  - human learning: based on observation and unsupervised interaction (action-perception loop)
- Issues
  - Training sets increasingly difficult to keep the pace with data production
  - Can we trust humans ?

“. . . hope is that deep unsupervised learning will be able to discover (possibly with a little bit of help from the few labeled examples we can provide) all of the concepts and underlying causes that matter (some being explicitly labeled, some remaining unnamed) to explain what we see around us. So I believe it is essential for approaching AI to make progress in this direction. And we are ;-)” Y. Bengio

Outline

1 Generative Adversarial Networks (GANs)

2 Auto-encoders for unsupervised learning

3 Attention based architectures

Attention is ubiquitous in DL



A woman is throwing a frisbee in a park.



A dog is standing on a hardwood floor.



A stop sign is on a road with a mountain in the background.



A little girl sitting on a bed with a teddy bear.



A group of people sitting on a boat in the water.

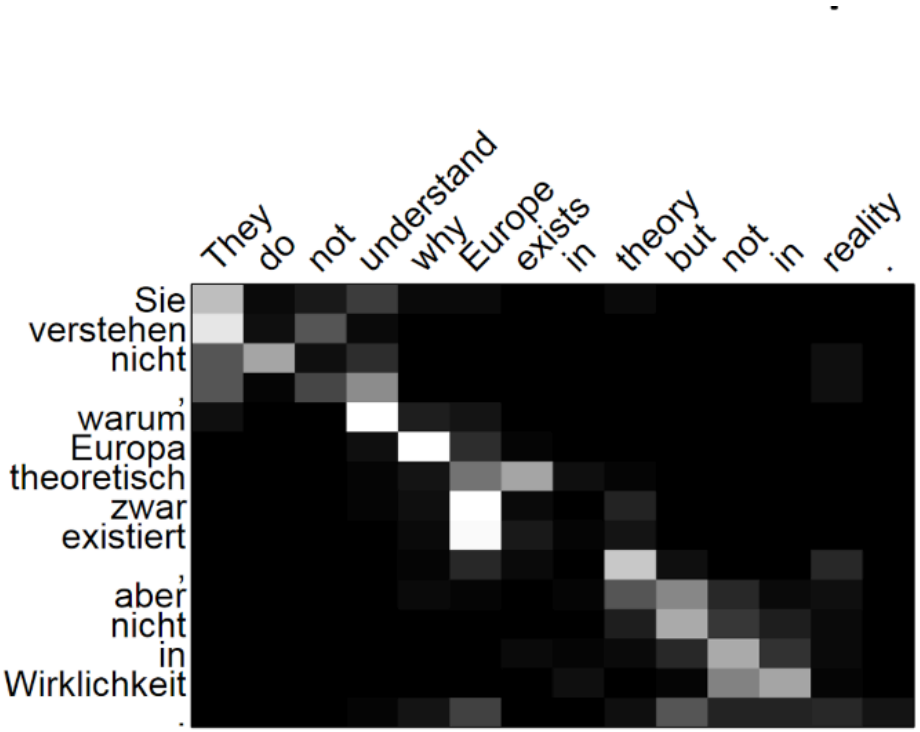


A giraffe standing in a forest with trees in the background.

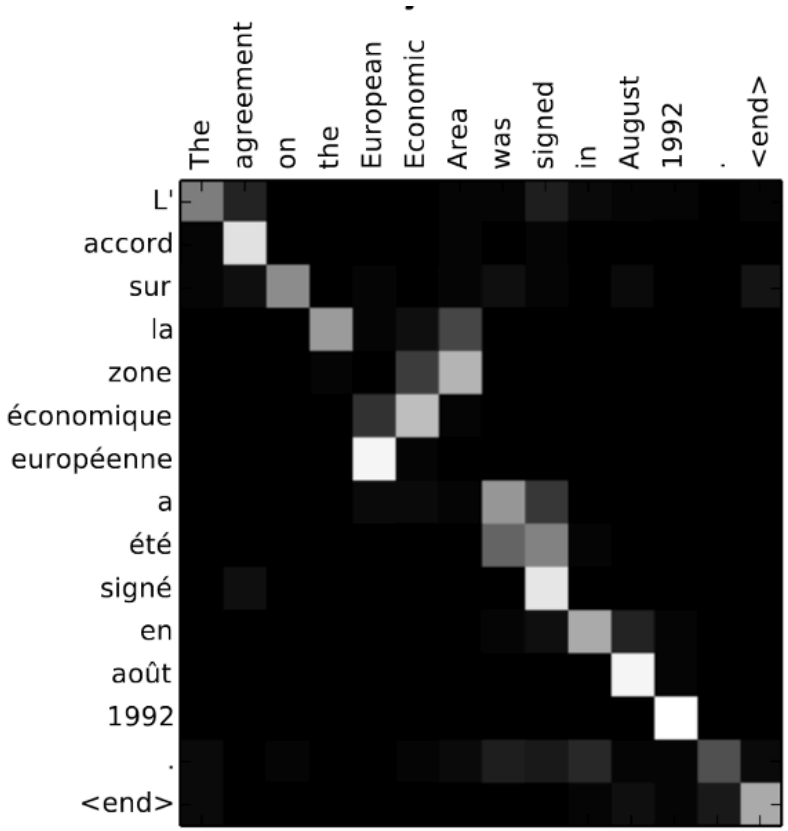
Show, attend and tell: Neural image caption generation with visual attention (Xu et al. 2015)

Attention is ubiquitous in DL

Neural machine translation



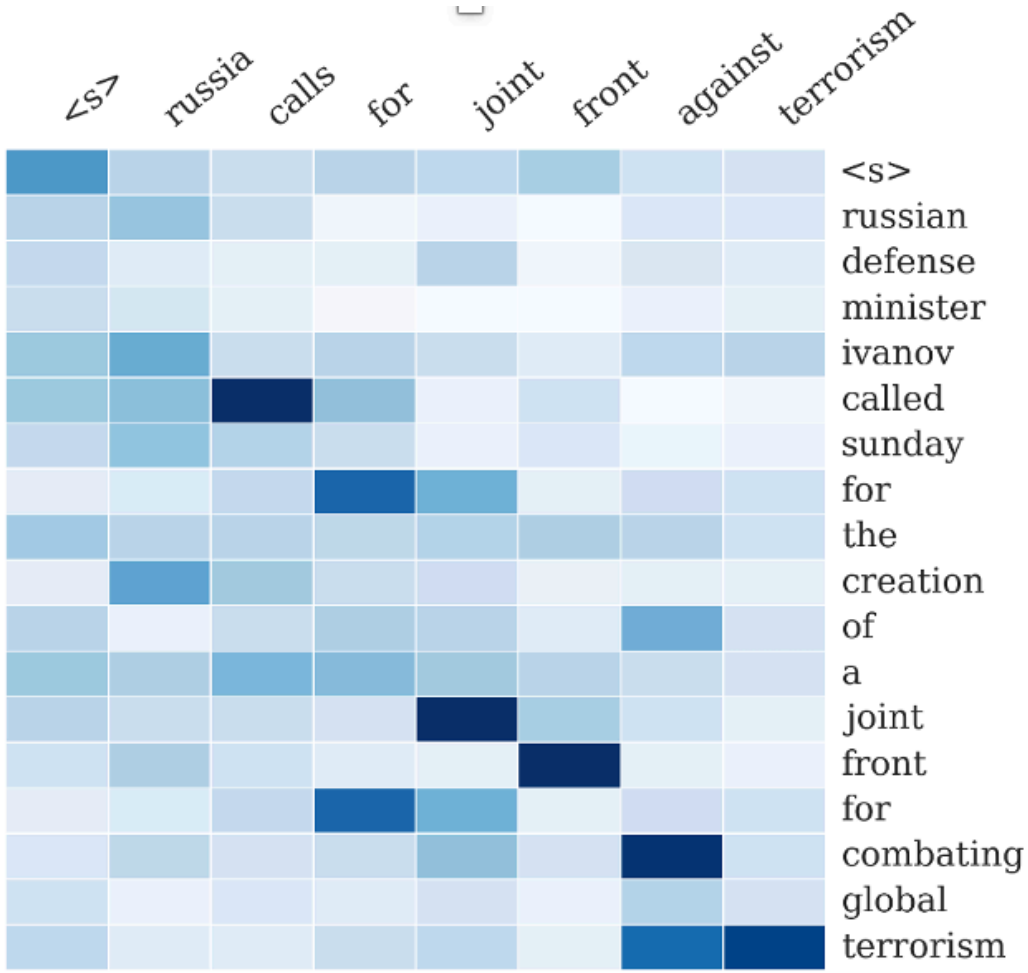
Effective approaches to attention-based neural machine translation (Luong et al. 2015)



Neural machine translation by jointly learning to align and translate (Bahdanau et al. 2014)

Attention is ubiquitous in DL

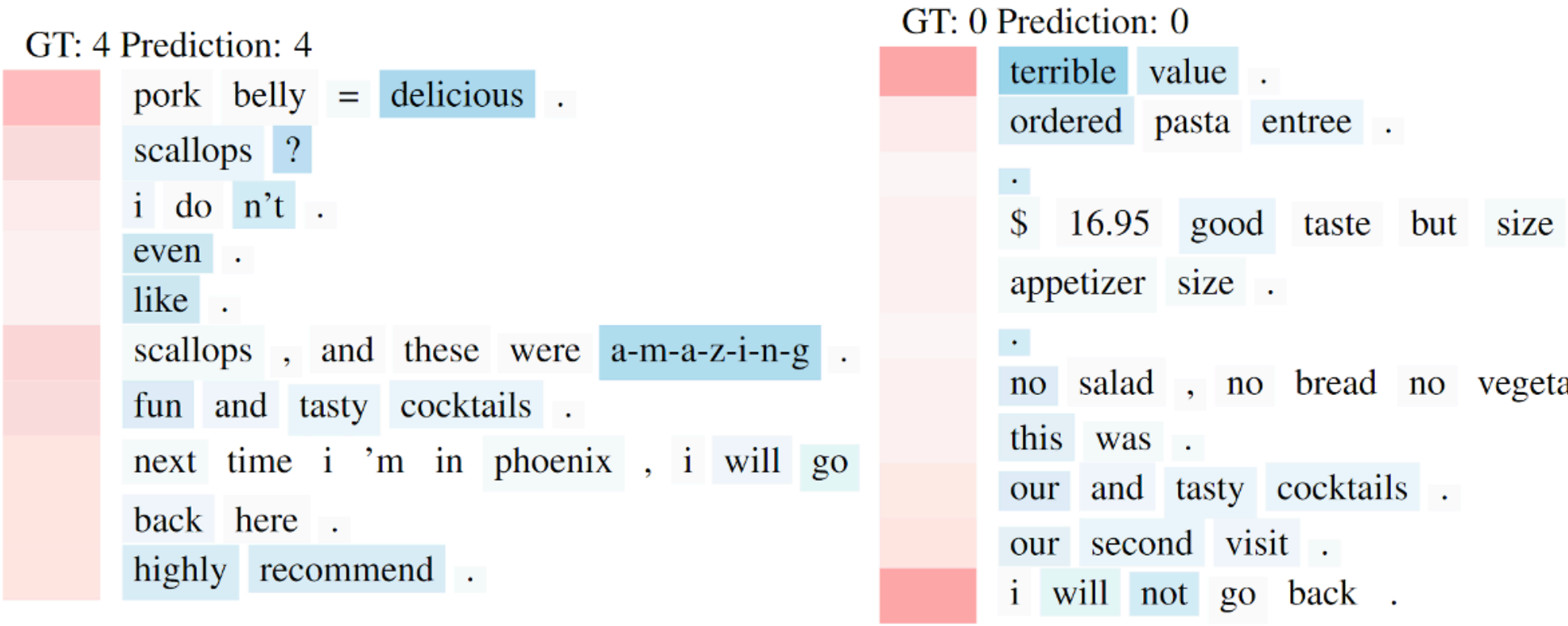
Abstractive summarization



A neural attention model for abstractive sentence summarization (Rush et al. 2015)

Attention is ubiquitous in DL

Sentiment analysis



Hierarchical attention networks for document classification (Yang et al. 2016)

Context Senses

- **”a word is defined by “the company it keeps” (Firth, 1957)**
- Words are **ambiguous**:
  - The amount of deposits in the **bank** decreased by 3.5% in March
  - The trees on **bank** of the river were offering their shade to the visitors
- We need **different vectors** to represent all word/token senses

Deep Contextualized Word Representations textsuperscript1

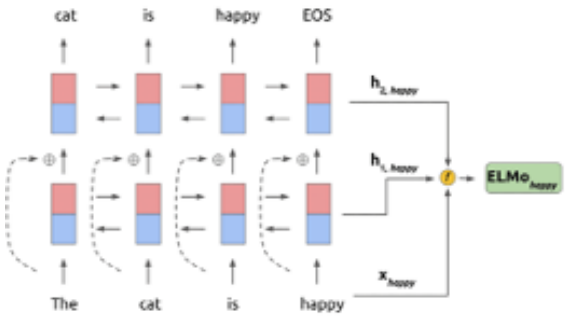
- **ELMo**: Embeddings from Language Models
- **Initial problem**: traditional word vectors map each word to a single, context-independent vector
- But some words have more than one sense! (**polysemy**)
  - e.g., bank, get, wood, play, mean...
- **Solution:**
  - A **token** is assigned a representation based on the **entire input sentence**
  - Use the **internal representations** of a **RNN language model** pre-trained on a large dataset



[1] Matthew E. Peters, Mark Neumann, Mohit Iyyer, Matt Gardner, Christopher Clark, Kenton Lee, and Luke Zettlemoyer. 2018. **Deep Contextualized Word Representations**. in *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics*; pages 2227–2237, New Orleans, Louisiana. Association for Computational Linguistics.

Deep Contextualized Word Representations

- **bidirectional LSTM** is trained with a coupled **language model (LM)** on a large text corpus - **ELMo** (Embeddings from Language Models) representations.



- **Forward**

$$p(t_1, t_2, \ldots, t_N) = \prod_{k=1}^{N} p(t_k | t_1, t_2, \ldots, t_{k-1})$$

- **Backward**

$$p(t_1, t_2, \ldots, t_N) = \prod_{k=1}^{N} p(t_k | t_{k+1}, t_{k+2}, \ldots, t_N)$$

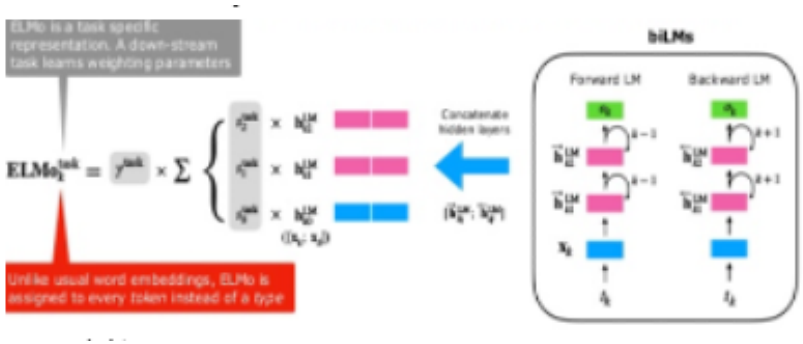
Graphic from: https://www.mihaileric.com/posts/deep-contextualized-word-representations-elmo/ “

Deep Contextualized Word Representations

“

Semi-supervised approach:

1. A deep bidirectional RNN language model is **pretrained on a large dataset**
2. The vector of each word in a given input sentence is computed as a **weighted sum of the RNN hidden states**
3. **1) is unsupervised, 2) weights are learned in a supervised way** on some task-specific dataset



ELMo Formulation

$$\text{ELMo}_k^{\text{task}} = \gamma^{\text{task}} \sum_{j=0}^{L} s_j^{\text{task}} \mathbf{h}_{k,j}^{\text{LM}}$$

- $\mathbf{h}_{k,j}^{\text{LM}}$ is the kth hidden representation of the jth layer of the bi-RNN LM
- s_j^{task} is the **softmax weight vector**, γ^{task} is a **scaling parameter** (for optimization)
- The authors use $\mathbf{L} = \mathbf{2}$ in all experiments

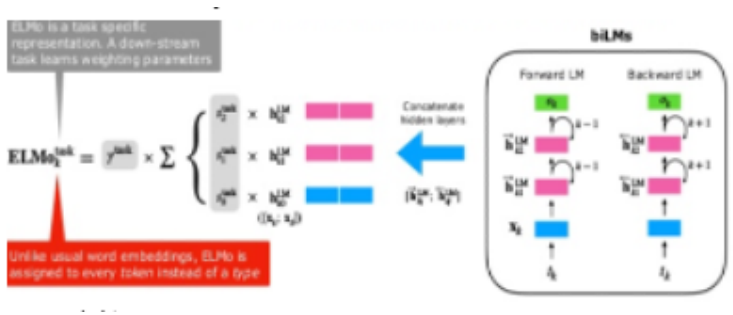
Graphic from: https://www.mihaileric.com/posts/deep-contextualized-word-representations-elmo/ “

Use of ELMo in Practice

Use of ELMo in practice, on a task-specific dataset:

1. Use the **pretrained language model** in prediction mode and store $\mathbf{h}_{k,j}^{\text{LM}}$ for each word (k) and each layer (j).
2. Concatenate $\mathbf{ELMo}_k^{\text{task}}$ with the corresponding **input vector*** of whatever supervised model is used to solve the task (e.g., RNN, CNN, feed-forward...)
3. Update s_j^{task} and γ^{task} with the other parameters of the supervised model during training.

*given by word2vec, glove, etc.



Results:

- ELMo improves over the baselines on **8 tasks**, ranging from question answering to co-reference resolution, sentiment analysis, POS-tagging, and disambiguation.

Deep Contextualized Word Representations - Experiments I

Task	Previous SOTA		Our baseline	ELMo + baseline	Increase (absolute/ relative)
SQuAD	Liu et al. (2017)	84.4	81.1	85.8	4.7 / 24.9%
SNLI	Chen et al. (2017)	88.6	88.0	88.7 ± 0.17	0.7 / 5.8%
SRL	He et al. (2017)	81.7	81.4	84.6	3.2 / 17.2%
Coref	Lee et al. (2017)	67.2	67.2	70.4	3.2 / 9.8%
NER	Peters et al. (2017)	91.93 ± 0.19	90.15	92.22 ± 0.10	2.06 / 21%
SST-5	McCann et al. (2017)	53.7	51.4	54.7 ± 0.5	3.3 / 6.8%

Table 1: Test set comparison of ELMo enhanced neural models with state-of-the-art single model baselines across six benchmark NLP tasks. The performance metric varies across tasks – accuracy for SNLI and SST-5; F_1 for SQuAD, SRL and NER; average F_1 for Coref. Due to the small test sizes for NER and SST-5, we report the mean and standard deviation across five runs with different random seeds. The "increase" column lists both the absolute and relative improvements over our baseline.

- SQuAD: Question answering, SNLI: Entailment, SRL: Semantic Role Labelling ("Who did what to whom"), NER: Name entity recognition, SST: Sentiment Analysis, Coref: task of clustering mentions in text that refer to the same underlying real world entities

Deep Contextualized Word Representations - Experiments II

Model	F_1
WordNet 1st Sense Baseline	65.9
Raganato et al. (2017a)	69.9
Iacobacci et al. (2016)	**70.1**
CoVe, First Layer	59.4
CoVe, Second Layer	*64.7*
biLM, First layer	67.4
biLM, Second layer	*69.0*

WSD

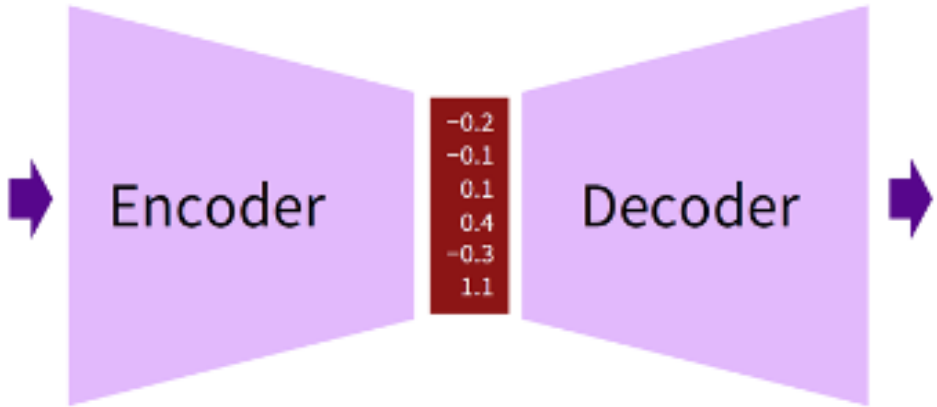
Model	Acc.
Collobert et al. (2011)	97.3
Ma and Hovy (2016)	97.6
Ling et al. (2015)	**97.8**
CoVe, First Layer	*93.3*
CoVe, Second Layer	92.8
biLM, First Layer	*97.3*
biLM, Second Layer	96.8

POS tagging

Encoder–Decoder Architectures

General idea: map input sequence to output sequence through encoder-decoder design.

- Encoder input: image, document, source language,...$\Rightarrow$ **embedding**
- Decoder Output: caption, summary, target language,...



Known as **sequence-to-sequence (seq2seq)** when input and output are sequences (e.g., NLP applications).
The full architecture is differentiable $\Rightarrow$ end-to-end training.

Objective and Background

Objective:

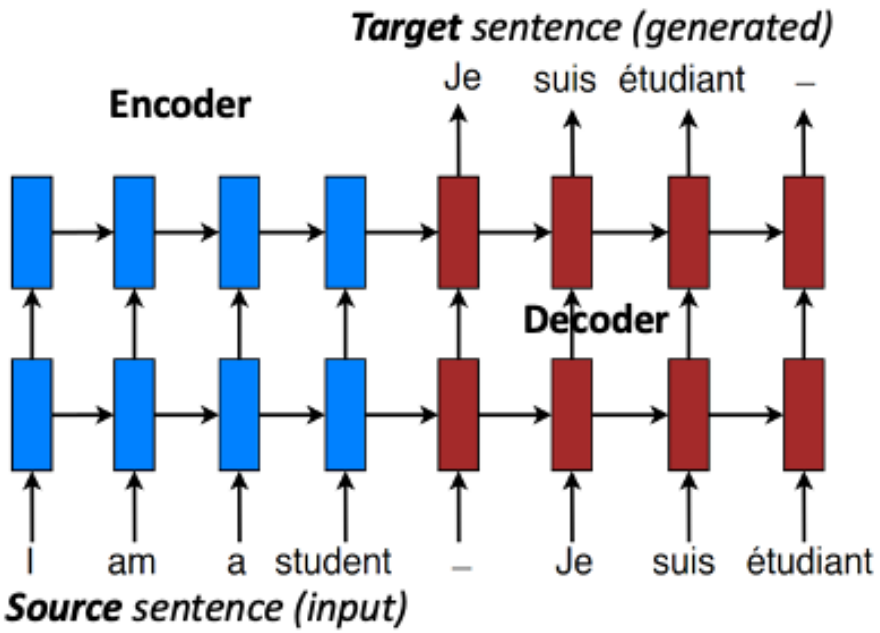
- Traditional models (e.g., encoder) embed the input into a single fixed-length vector (lossy).
- Alternatively, information can be stored into multiple vectors.
- Information can be retrieved later (e.g., by the decoder). *(Bahdanau et al., 2014)*

Quick History:

- Developed in the context of neural machine translation (Bahdanau et al., 2014).
- Applied to image captioning (Xu et al., 2015) and summarization (Luong et al., 2015).
- Proposed for encoders only: text classification (Yang et al., 2015) and representation learning (Conneau et al., 2017).

Known as **self or inner attention** in such cases.

Encoder–Decoder for Neural Machine Translation



- Source: $(x_1, \ldots, x_{T_x})$,
- Target: $(y_1, \ldots, y_{T_y})$
- Encoder & decoder: unidirectional deep RNNs (a.k.a. stacking RNNs)
- Training objective:

$$\arg\max_{\theta} \left\{ \sum_{(x,y)\in \text{corpus}} \log p(y \mid x; \theta) \right\}$$

Effective Approaches to Attention-Based Neural Machine Translation (Luong et al., 2015)
https://arxiv.org/pdf/1508.04025.pdf

Encoder–Decoder Details

Encoder:

- Usually: CNN, or stacking RNNs with LSTM or GRU units.
- Unidirectional (Luong et al., 2015) or bidirectional (Bahdanau et al., 2014).

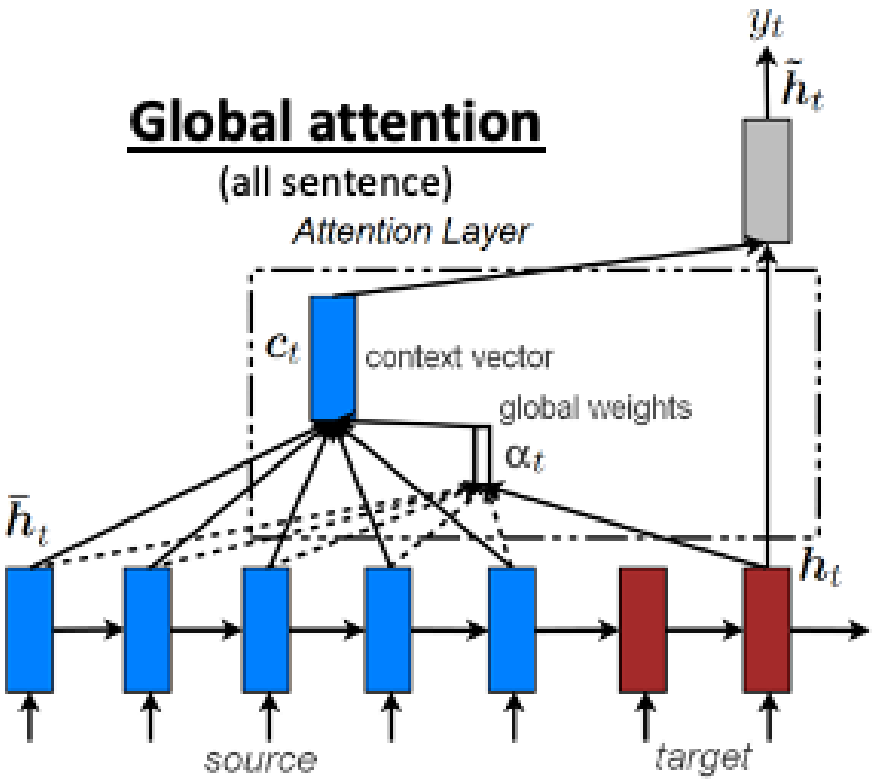
Decoder:

- Unidirectional RNN (well-suited to text generation), best if deep.
- Generates the target sentence $(y_1, \ldots, y_{T_y})$ one word at a time:

$$P[y_t \mid \{y_1, \ldots, y_{t-1}\}, c_t] = \text{softmax}\Big(W_s \tilde{h}_t\Big)$$

Effective Approaches to Attention-Based Neural Machine Translation (Luong et al., 2015)
https://arxiv.org/pdf/1508.04025.pdf

Attention Mechanism - Global



$$P\left[y_t|\{y_1,\ldots,y_{t-1}\},c_t\right] = \text{softmax}\left(W_s\tilde{h}_t\right)$$

attentional hidden state

$$\tilde{h}_t = \tanh\left(W_c\left[c_t;h_t\right]\right)$$

context vector

decoder hidden state

$$c_t = \sum_{i=1}^{T_x} \alpha_{t,i}\bar{h}_i$$

i^{th} encoder hidden state

alignment vector

$$\alpha_{t,i} = \frac{\exp\left(\text{score}\left(h_t,\bar{h}_i\right)\right)}{\sum_{i'=1}^{T_x}\exp\left(\text{score}\left(h_t,\bar{h}_{i'}\right)\right)}$$

$$\text{score}\left(h_t,\bar{h}_i\right) = h_t^\top\bar{h}_i$$

Global Attention Mechanism - Example

Source Sentence: I $\rightarrow$ am $\rightarrow$ a $\rightarrow$ student
Encoder Hidden States:

$$\bar{h}_1, \bar{h}_2, \bar{h}_3, \bar{h}_4$$

(One for each word)

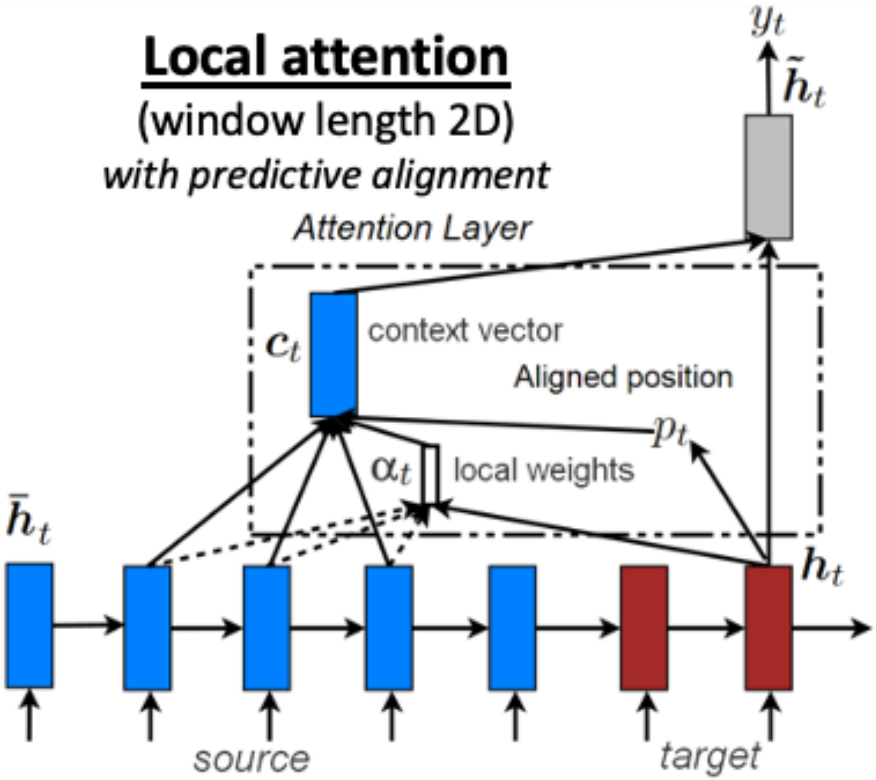
- **Global Attention** uses **all** source hidden states ($\{\bar{h}_i\}_{i=1}^{N}$) to compute context vector (c_t) for *each* target word (t) - looks entire source sentence at each decoding step.
- **Generating Je ($t = 1$):**
  - Target state h_1 generated.
  - **Scores (α_{1i})** computed between h_1 and all states ($\bar{h}_1, \ldots, \bar{h}_4$).
  - Je predicted, with $\bar{\mathbf{h}}_1$ (I) highest weight.
- **Generating Je ($t = 1$):**
  - Target state h_1 generated - **Scores (α_{1i})** computed between h_1 and all states ($\bar{h}_1, \ldots, \bar{h}_4$).
  - Je predicted with $\bar{\mathbf{h}}_1$ (I) highest weight.
- **Generating suis ($t = 2$):**
  - Target state h_2 generated; new scores (α_{2i}) computed between h_2 and all states.
  - suis predicted, with $\bar{\mathbf{h}}_2$ (am) highest weight.
- **Generating étudiant ($t = 3$):**
  - Target state h_3 generated - new scores (α_{3i}) computed between h_3 and all states.
  - étudiant predicted, with $\bar{\mathbf{h}}_4$ (student) highest weight.

Mathematical Formulation (Context Vector c_t)

Attention Weights: $\alpha_{ti} = \frac{\exp(\text{score}(h_t, \bar{h}_i))}{\sum_{k=1}^{N} \exp(\text{score}(h_t, \bar{h}_k))}$

Context Vector: $c_t = \sum_{i=1}^{N} \alpha_{ti} \bar{h}_i$

Attention Mechanism - Local



$$P\left[y_t|\{y_1,...,y_{t-1}\},c_t\right] = \text{softmax}\left(W_s\tilde{h}_t\right)$$

attentional hidden state

$$\tilde{h}_t = \tanh\left(W_c\left[c_t;h_t\right]\right)$$

decoder hidden state

$$p_t = T_x \cdot \sigma\left(v_p^\top \tanh(W_p h_t)\right)$$

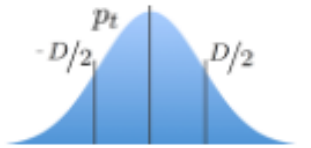
context vector

$$c_t = \sum_{i=p_t-D}^{p_t+D} \alpha_{t,i}\bar{h}_i$$

i^{th} encoder hidden state

alignment vector

$$\alpha_{t,i} = \frac{\exp\left(\text{score}(h_t,\bar{h}_i)\right)}{\sum_{i'=p_t-D}^{p_t+D}\exp\left(\text{score}(h_t,\bar{h}_{i'})\right)}\exp\left(-\frac{(i-p_t)^2}{2(D/2)^2}\right)$$



$$\text{score}(h_t,\bar{h}_i) = h_t^\top W_\alpha \bar{h}_i$$

Luong et al. (2015)

Key Insights from the Encoder-Decoder Model

- Tested on the English $\leftrightarrow$ German task (WMT'14 dataset)
- 4.5M sentence pairs
- Encoder and decoder RNNs feature 4 layers of stacking and 1000-dimensional hidden states
- Window of size $D = 10$ for local attention
- Trained for 12 epochs (total of 7-10 days on a single GPU, at 1K target words/s)
- New state-of-the-art performance

- **Local attention** with predictive alignment gives better results than **global attention**
- **Dot product** (score$(h_t, \bar{h}_i) = h_t^\top \bar{h}_i$) works well for **global attention**
- The **general formulation** (score$(h_t, \bar{h}_i) = h_t^\top W_a \bar{h}_i$) is better for **local attention**

Conclusions

- Gan architectures
- Autoencoders
- Attention based architectures

Next class

- Transformers
- LLM architecture

References

1. **Autoencoders (AEs)**
   - **Deep Autoencoders (Pre-training):** Hinton, G. E., & Salakhutdinov, R. R. (2006). Reducing the dimensionality of data with neural networks. *Science, 313*(5786), 504-507.
   - **Variational Autoencoders (VAEs):** Kingma, D. P., & Welling, M. (2014). Auto-Encoding Variational Bayes. *(ICLR)*.
2. **Generative Adversarial Networks (GANs)**
   - **Foundational GAN:** Goodfellow, I., Pouget-Abadie, J., et al. (2014). Generative Adversarial Networks. *(NeurIPS), 27*.
   - **Convolutional GANs (DCGAN):** Radford, A., Metz, L., & Chintala, S. (2016). Unsupervised Representation Learning with Deep Convolutional Generative Adversarial Networks. *(ICLR)*.
3. **Attention Mechanism in AI**
   - **Initial Soft Attention (Seq2Seq):** Bahdanau, D., Cho, K., & Bengio, Y. (2014). Neural Machine Translation by Jointly Learning to Align and Translate. *(ICLR)*.
   - **Transformer / Self-Attention:** Vaswani, A., Shazeer, N., Parmar, N., et al. (2017). Attention Is All You Need. *(NeurIPS), 30*.